

如果你认为 `std::iterator_traits<IterT>::value_type` 读起来不畅快, 想象一下打那么长的字又是什么光景。如果你像大多数程序员一样, 认为多打几次这些字实在很恐怖, 那么你应该会想建立一个 `typedef`。对于 `traits` 成员名称如 `value_type` (再次请看条款 47 提供的 `traits` 信息), 普遍的习惯是设定 `typedef` 名称用以代表某个 `traits` 成员名称, 于是常常可以看到类似这样的 `local typedef`:

```
template<typename IterT>
void workWithIterator(IterT iter)
{
    typedef typename std::iterator_traits<IterT>::value_type value_type;
    value_type temp(*iter);
    ...
}
```

许多程序员最初认为把 "typedef typename" 并列颇不合谐, 但它实在是指涉“嵌套从属类型名称”的一个合理附带结果。你很快会习惯它, 毕竟你有强烈的动机——你希望多打几次 `typename std::iterator_traits<IterT>::value_type` 吗?

作为结语, 我应该提到, `typename` 相关规则在不同的编译器上有不同的实践。某些编译器接受的代码原本该有 `typename` 却遗漏了; 原本不该有 `typename` 却出现了; 还有少数编译器 (通常是较旧版本) 根本就拒绝 `typename`。这意味 `typename` 和“嵌套从属类型名称”之间的互动, 也许会在移植性方面带给你某种温和的头疼。

请记住

- 声明 `template` 参数时, 前缀关键字 `class` 和 `typename` 可互换。
- 请使用关键字 `typename` 标识嵌套从属类型名称; 但不得在 `base class lists` (基类列) 或 `member initialization list` (成员初值列) 内以它作为 `base class` 修饰符。

条款 43: 学习处理模板化基类内的名称

Know how to access names in templated base classes.

假设我们需要撰写一个程序, 它能够传送信息到若干不同的公司去。信息要不译成密码, 要不就是未经加工的文字。如果编译期间我们有足够信息来决定哪一个信息传至哪一家公司, 就可以采用基于 `template` 的解法:

```

class CompanyA {
public:
    ...
    void sendCleartext(const std::string& msg);
    void sendEncrypted(const std::string& msg);
    ...
};

class CompanyB {
public:
    ...
    void sendCleartext(const std::string& msg);
    void sendEncrypted(const std::string& msg);
    ...
};

... //针对其他公司设计的 classes.

class MsgInfo { ... }; //这个 class 用来保存信息, 以备将来产生信息

template<typename Company>
class MsgSender {
public:
    ... //构造函数、析构函数等等.
    void sendClear(const MsgInfo& info)
    {
        std::string msg;
        在这儿, 根据 info 产生信息;
        Company c;
        c.sendCleartext(msg);
    }
    void sendSecret(const MsgInfo& info) //类似 sendClear, 唯一不同是
    { ... } //这里调用 c.sendEncrypted
};

```

这个做法行得通。但假设我们有时候想要在每次送出信息时记录(log)某些信息。derived class 可轻易加上这样的生产力, 那似乎是个合情合理的解法:

```

template<typename Company>
class LoggingMsgSender: public MsgSender<Company> {
public:
    ... //构造函数、析构函数 等等.
    void sendClearMsg(const MsgInfo& info)
    {
        将“传送前”的信息写至 log;
        sendClear(info); //调用 base class 函数; 这段码无法通过编译。
        将“传送后”的信息写至 log;
    }
    ...
};

```

注意这个 `derived class` 的信息传送函数有一个不同的名称 (`sendClearMsg`)，与其 `base class` 内的名称 (`sendClear`) 不同。那是个好设计，因为它避免遮掩“继承而得的名词”(见条款 33)，也避免重新定义一个继承而得的 `non-virtual` 函数(见条款 36)。但上述代码无法通过编译，至少对严守规律的编译器而言。这样的编译器会抱怨 `sendClear` 不存在。我们的眼睛可以看到 `sendClear` 的确在 `base class` 内，编译器却看不到它们。为什么？

问题在于，当编译器遭遇 `class template LoggingMsgSender` 定义式时，并不知道它继承什么样的 `class`。当然它继承的是 `MsgSender<Company>`，但其中的 `Company` 是个 `template` 参数，不到后来(当 `LoggingMsgSender` 被具现化)无法确切知道它是什么。而如果不知道 `Company` 是什么，就无法知道 `class MsgSender<Company>` 看起来像什么——更明确地说是没办法知道它是否有个 `sendClear` 函数。

为了让问题更具体化，假设我们有个 `class CompanyZ` 坚持使用加密通讯：

```
class CompanyZ {                                //这个 class 不提供
public:                                         //sendCleartext 函数
    ...
    void sendEncrypted(const std::string& msg);
    ...
};
```

一般性的 `MsgSender` `template` 对 `CompanyZ` 并不合适，因为那个 `template` 提供了一个 `sendClear` 函数(其中针对其类型参数 `Company` 调用了 `sendCleartext` 函数)，而这对 `CompanyZ` 对象并不合理。欲矫正这个问题，我们可以针对 `CompanyZ` 产生一个 `MsgSender` 特化版：

```
template<>                                     //一个全特化的
class MsgSender<CompanyZ> {                   //MsgSender; 它和一般 template 相同,
public:                                       //差别只在于它删掉了 sendClear。
    ...
    void sendSecret(const MsgInfo& info)
    { ... }
};
```

注意 `class` 定义式最前头的 `"template<>"` 语法象征这既不是 `template` 也不是标准 `class`，而是个特化版的 `MsgSender` `template`，在 `template` 实参是 `CompanyZ` 时被使用。这是所谓的模板全特化(*total template specialization*)：`template MsgSender` 针对类型 `CompanyZ` 特化了，而且其特化是全面性的，也就是说一旦类型参数被定义为 `CompanyZ`，再没有其他 `template` 参数可供变化。

现在, `MsgSender` 针对 `CompanyZ` 进行了全特化, 让我们再次考虑 `derived class` `LoggingMsgSender`:

```
template<typename Company>
class LoggingMsgSender: public MsgSender<Company> {
public:
    ...
    void sendClearMsg(const MsgInfo& info)
    {
        将“传送前”的信息写至 log;

        sendClear(info);          //如果 Company == CompanyZ, 这个函数不存在。

        将“传送后”的信息写至 log;
    }
    ...
};
```

正如注释所言, 当 `base class` 被指定为 `MsgSender<CompanyZ>` 时这段代码不合法, 因为那个 `class` 并未提供 `sendClear` 函数! 那就是为什么 C++ 拒绝这个调用的原因: 它知道 `base class templates` 有可能被特化, 而那个特化版本可能不提供和一般性 `template` 相同的接口。因此它往往拒绝在 `templated base classes` (模板化基类, 本例的 `MsgSender<Company>`) 内寻找继承而来的名称 (本例的 `SendClear`)。就某种意义而言, 当我们从 `Object Oriented C++` 跨进 `Template C++` (见条款 1), 继承就不像以前那般畅行无阻了。

为了重头来过, 我们必须有某种办法令 C++ “不进入 `templated base classes` 观察”的行为失效。有三个办法, 第一是在 `base class` 函数调用动作之前加上 `"this->":`

```
template<typename Company>
class LoggingMsgSender: public MsgSender<Company> {
public:
    ...
    void sendClearMsg(const MsgInfo& info)
    {
        将“传送前”的信息写至 log;

        this->sendClear(info);      //成立, 假设 sendClear 将被继承。

        将“传送后”的信息写至 log;
    }
    ...
};
```

第二是使用 using 声明式。如果你已读过条款 33, 这个解法应该会令你感到熟悉。条款 33 描述 using 声明式如何将“被掩盖的 base class 名称”带入一个 derived class 作用域内。我们可以这样写下 sendClearMsg:

```
template<typename Company>
class LoggingMsgSender: public MsgSender<Company> {
public:
    using MsgSender<Company>::sendClear;    //告诉编译器, 请它假设
    ...                                     //sendClear 位于 base class 内。
    void sendClearMsg(const MsgInfo& info)
    {
        ...
        sendClear(info);                    //OK, 假设 sendClear 将被继承下来。
        ...
    }
    ...
};
```

(虽然 using 声明式在这里或在条款 33 都可有效运作, 但两处解决的问题其实不相同。这里的情况并不是 base class 名称被 derived class 名称遮掩, 而是编译器不进入 base class 作用域内查找, 于是我们通过 using 告诉它, 请它那么做。)

第三个做法是, 明白指出被调用的函数位于 base class 内:

```
template<typename Company>
class LoggingMsgSender: public MsgSender<Company> {
public:
    ...
    void sendClearMsg(const MsgInfo& info)
    {
        ...
        MsgSender<Company>::sendClear(info);    //OK, 假设 sendClear
        ...                                     //将被继承下来。
    }
    ...
};
```

但这往往是最不让人满意的一个解法, 因为如果被调用的是 virtual 函数, 上述的明确资格修饰 (explicit qualification) 会关闭“virtual 绑定行为”。

从名称可视点 (visibility point) 的角度出发, 上述每一个解法做的事情都相同: 对编译器承诺“base class template 的任何特化版本都将支持其一般 (泛化) 版本所提供的接口”。这样一个承诺是编译器在解析 (parse) 像 LoggingMsgSender 这样的 derived class template 时需要的。但如果这个承诺最终未被实践出来, 往后的编

译最终还是会还给事实一个公道。举个例子，如果稍后的源码内含这个：

```
LoggingMsgSender<CompanyZ> zMsgSender;
MsgInfo msgData;
...                               //在 msgData 内放置信息。
zMsgSender.sendClearMsg(msgData); //错误！无法通过编译。
```

其中对 `sendClearMsg` 的调用动作将无法通过编译，因为在那个点上，编译器知道 **base class** 是个 **template** 特化版本 `MsgSender<CompanyZ>`，而且它们知道那个 **class** 不提供 `sendClear` 函数，而后者却是 `sendClearMsg` 尝试调用的函数。

根本而言，本条款探讨的是，面对“指涉 **base class members**”之无效 **references**，编译器的诊断时间可能发生在早期（当解析 **derived class template** 的定义式时），也可能发生在晚期（当那些 **templates** 被特定之 **template** 实参具现化时）。C++ 的政策是宁愿较早诊断，这就是为什么“当 **base classes** 从 **templates** 中被具现化时”它假设它对那些 **base classes** 的内容毫无所悉的缘故。

请记住

- 可在 **derived class templates** 内通过 `"this->"` 指涉 **base class templates** 内的成员名称，或藉由一个明白写出的“**base class** 资格修饰符”完成。

条款 44：将与参数无关的代码抽离 **templates**

Factor parameter-independent code out of templates.

Templates 是节省时间和避免代码重复的一个奇方妙法。不再需要键入 20 个类似的 **classes** 而每一个带有 15 个成员函数，你只需键入一个 **class template**，留给编译器去具现化那 20 个你需要的相关 **classes** 和 300 个函数。（**class templates** 的成员函数只有在被使用时才被暗中具现化，所以只有在这 300 个函数的每一个都被使用，你才会获得这 300 个函数。）**Function templates** 有类似的诉求。替换写许多函数，你只需要写一个 **function template**，然后让编译器做剩余的事情。技术是不是很高伟大，呵呵。

是的，唔……有时候啦。如果你不小心，使用 **templates** 可能会导致代码膨胀（*code bloat*）：其二进制码带着重复（或几乎重复）的代码、数据，或两者。其结果有可能源码看起来合身而整齐，但目标码（**object code**）却不是那么回事。肥胖不结实很难被视为时尚，所以你需要知道如何避免这样的二进制浮夸。

你的主要工具有个气势恢宏的名称：共性与变性分析（*commonality and variability analysis*），但其概念十分平民化。纵使你从未写过一个 **template**，你始